



9.1. Regularizacija i regularizirana LR

- trošak = gubitak (pogreške) + složenost
- Cost = loss + complexity
- Regularizacija - održavanje niske složenosti i jednostavnosti modela
- $y = w_2x_2 + w_1x_1 + w_0$
- Kako smanjiti složenost:
 - a) Postavljanjem težina na 0
 - b) Postavljanjem težina na male vrijednosti ! - REGULARIZACIJA



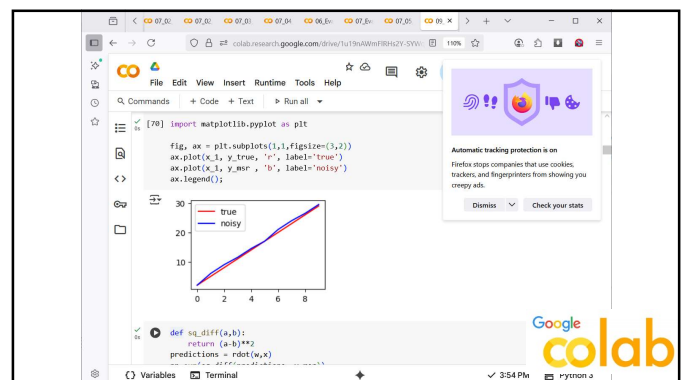
```
from sklearn import datasets
from sklearn import model_selection as skms
import matplotlib inline

diabetes = datasets.load_diabetes()
d_tts = skms.train_test_split(diabetes.data,
                              diabetes.target,
                              test_size=.25,
                              random_state=42)
(diabetes_train_ftrs, diabetes_test_ftrs,
 diabetes_train_tgt, diabetes_test_tgt) = d_tts

[62] import numpy as np
weights = np.array([3.5, -2.1, .7])
print(np.sum(np.abs(weights)),
      np.sum(weights**2))

6.3 17.15

def rdot(arr,br):
    return np.dot(br,arr)
```



```
np.float64(5.8)

predictions = rdot(w,x)

loss = np.sum(sq_diff(predictions, y_nr))
complexity_1 = np.sum(np.abs(weights))
complexity_2 = np.sum(weights**2) # = np.dot(weights, weights)
print(complexity_1)
print(complexity_2)
print(loss)
cost_1 = loss + complexity_1
cost_2 = loss + complexity_2

print("Sum(abs) complexity:", cost_1)
print("Sum(sq) complexity:", cost_2)

6.3
17.15
5.8
Sum(abs) complexity: 11.3
Sum(sq) complexity: 22.15
```

- $cost_1 = loss + complexity_1$
- $cost_2 = loss + complexity_2$



```

print(complexity_2)
print(loss)
cost_1 = loss + complexity_1
cost_2 = loss + complexity_2

print("Sum(abs) complexity:", cost_1)
print("Sum(sq) complexity:", cost_2)

6.3
17.15
5.8
Sum(abs) complexity: 11.3
Sum(sq) complexity: 22.15

[32] predictions = dot(w,x)
errors = np.sum(sq_diff(predictions, y_test))
complexity_1 = np.sum(np.abs(weights))

C = .5
cost = errors + C * complexity_1
cost

np.float64(8.15)

```

- Cost1 - L1-regularizirana regresija ili *lasso regression*
- Cost2 – L2-regularizirana regresija ili *ridge regression*

$$\text{Cost}_1 = \sum_{x,y \in D} (wx - y)^2 + C \sum_j |w_j|$$

$$\text{Cost}_2 = \sum_{x,y \in D} (wx - y)^2 + C \sum_j w_j^2$$

```

models = [linear_model.Lasso(), # L1 regularizirana; C=1.0
          linear_model.Ridge()] # L2 regularizirana; C=1.0

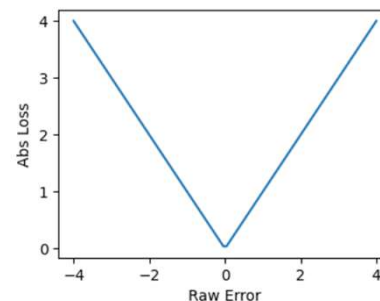
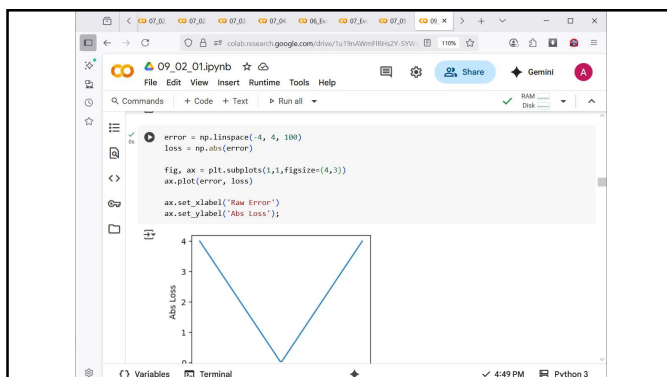
for model in models:
    model.fit(diabetes_train_ftrs, diabetes_train_tgt)
    train_preds = model.predict(diabetes_train_ftrs)
    test_preds = model.predict(diabetes_test_ftrs)
    print(f"Model: {model.__class__.__name__}")
    print(f"Train MSE: {metrics.mean_squared_error(diabetes_train_tgt, train_preds)}")
    print(f"Test MSE: {metrics.mean_squared_error(diabetes_test_tgt, test_preds)}")

Lasso
Train MSE: 3947.9853297148935
Test MSE: 3433.155492119971
Ridge
Train MSE: 3461.743744823881
Test MSE: 3105.4721464484733

```

9.2. SVR – Support Vector Regression (Regresija pomoću potpornih vektora)

- trošak = gubitak + složenost
- U SVR koristimo apsolutne vrijednosti pogrešaka, ali male vrijednosti **zamenarujemo!**



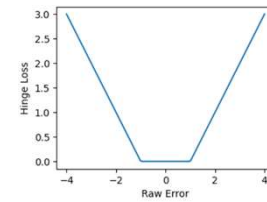
$$\bullet \text{ loss} = \max(|\text{error}| - \text{threshold}; 0)$$

```

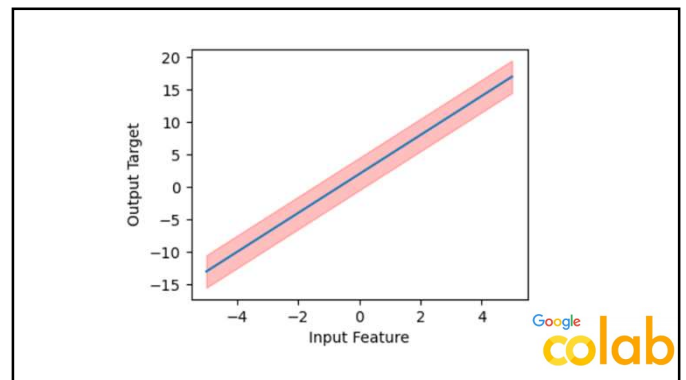
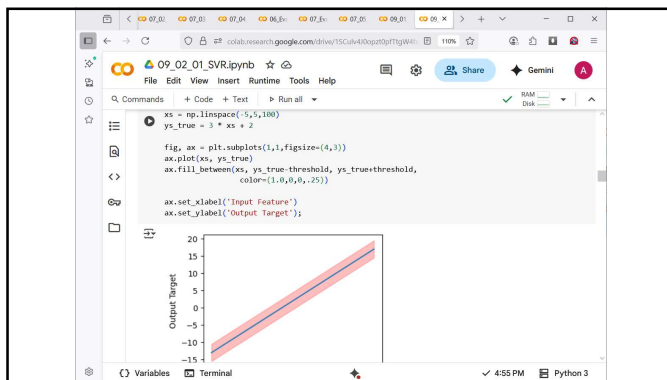
1 # Hinge Loss
2 def hinge_loss(error, threshold):
3     loss = max(abs(error) - threshold, 0)
4     return loss
5
6 # Example usage
7 error = 3.5
8 threshold = 2.5
9 loss = hinge_loss(error, threshold)
10 print(loss)

```

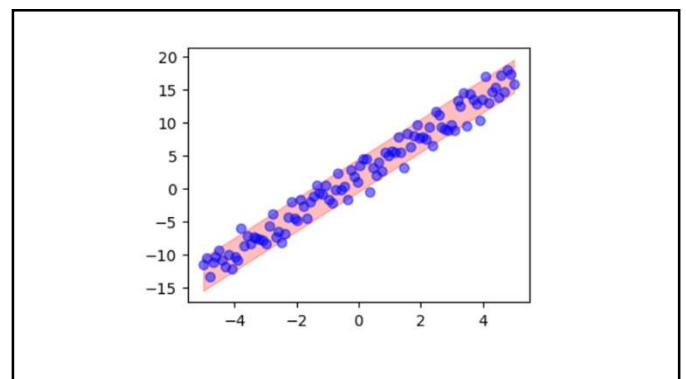
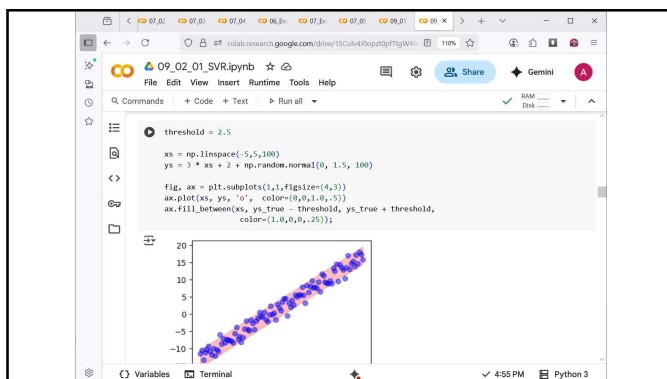
$$\bullet \text{ loss} = \max(|\text{error}| - \text{threshold}; 0)$$



Google
colab



Google
colab





```

09_02_01_SVR.ipynb
File Edit View Insert Runtime Tools Help
[17] # hyperparameter
C, epsilon = 1.0, .25

# parameter
weights = np.array([1.3])

# prediction, error, loss
def rdot(arr, arr1):
    return np.dot(arr, arr1)
predictions = rdot(weights, xs.reshape(-1, 1))
errors = ys - predictions

loss_sse = np.sum(errors ** 2)
loss_sse = np.sum(np.abs(errors))
loss_hinge = np.sum(np.max(np.abs(errors) - epsilon, 0))

[19] # complexity penalty for regularization
complexity_saw = np.sum(np.abs(weights))
complexity_ssw = np.sum(weights**2)

[21] # cost

```

```

print(cost_gof_regression, cost_l1pen_regression, cost_l2pen_regression, cost_sv_regression)
3249.9372804843754 3251.2372004843755 3251.6272004843754 15.724956639152678

```

Name	Penalty	Math
GOF Linear Regression	None	$\sum_i (y_i - wx_i)^2$
Lasso	L_1	$\sum_i (y_i - wx_i)^2 + C \sum_j w_j$
Ridge	L_2	$\sum_i (y_i - wx_i)^2 + C \sum_j w_j^2$
SVR	L_1	$\sum_i \max(y_i - wx_i - \epsilon, 0) + C \sum_i w_j$

Name	Loss	Penalty
GOF LR	L_2	0
Lasso (L_1 -Penalty LR)	L_2	L_1
Ridge (L_2 -Penalty LR)	L_2	L_2
SVR	Hinge	L_1

- ϵ -SVR - toleranciju opsega pogreške
- ν -SVR - udio zadržanih vektora podrške s obzirom na ukupan broj primjera

```

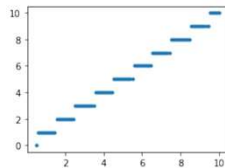
[25] # cost
cost_gof_regression = loss_sse + 0.0
cost_l1pen_regression = loss_sse + C * complexity_saw
cost_l2pen_regression = loss_sse + C * complexity_ssw
cost_sv_regression = loss_hinge + C * complexity_ssw
print(cost_gof_regression, cost_l1pen_regression, cost_l2pen_regression, cost_sv_regression)
3249.9372804843754 3251.2372004843755 3251.6272004843754 15.724956639152678

from sklearn import svm
from sklearn import metrics
svrs = [svm.SVR(), # default epsilon=0.1
        svm.NuSVR()] # default nu=0.5

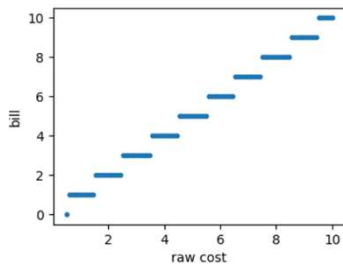
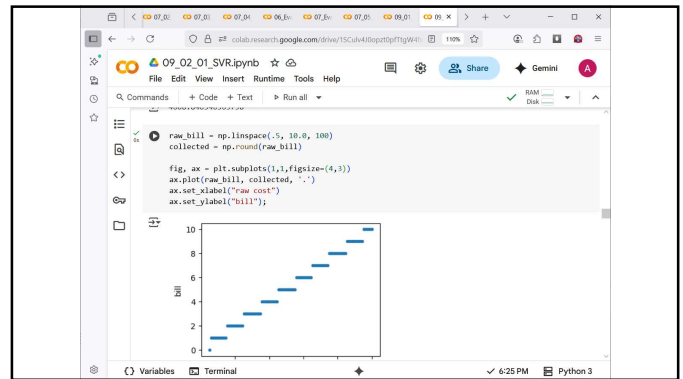
for model in svrs:
    preds = (model.fit(diabetes_train_fts, diabetes_train_tgt)
             .predict(diabetes_test_fts))
    print(metrics.mean_squared_error(diabetes_test_tgt, preds))
4511.870338681654
4608.840546363738

```

9.3 Djelomično konstantna regresija



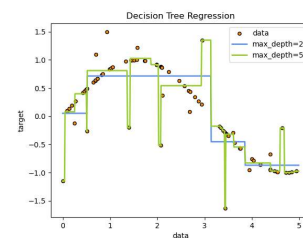
Google
colab



• Sklearn dokumenti govore o tome kako implementirati prilagođene modele:

- Budući da implementiramo regresor, naslijedit ćemo BaseEstimator i RegressorMixin.
- Nećemo ništa učiniti s argumentima modela u `_init__`.
- Definirat ćemo metode `fit(X,y)` i `predict(X)`
- Možemo koristiti `check_X_y` i `check_array` za provjeru jesu li argumenti važeći za sklearn

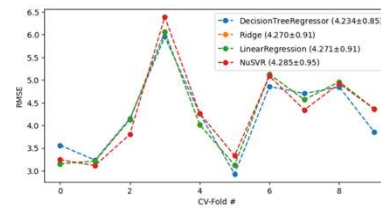
9.3. Stabla regresije



Google
colab

DecisionTreeRegressor 1 4341
 DecisionTreeRegressor 3 3593
 DecisionTreeRegressor 5 4312
 DecisionTreeRegressor 10 5190

9.4. Usporedba regresora



Google
colab

